

AFRILANG Stdlib

std/process

Français. Bibliothèque AFRILANG 'std/process' (niveau simple, catégorie system). Elle expose 2 fonction(s) : exitOk, exitFailed.

'import "std/process"' · 'use process'

- 'exitOk(code number) → bool' - 'exitFailed(code number) → bool' **En-**

glish. AFRILANG library 'std/process' (tier simple, category system).

Exports 2 function(s): exitOk, exitFailed.

'import "std/process"' · 'use process'

- 'exitOk(code number) → bool' - 'exitFailed(code number) → bool'

Catégorie / Category	Système	
Niveau / Tier	simple	June 16, 2026
Fonctions / Functions	2	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/process' (niveau simple, catégorie system). Elle expose 2 fonction(s) : exitOk, exitFailed.

'import "std/process"' · 'use process'

```
- `exitOk(code number) → bool`
- `exitFailed(code number) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/process"
use process
```

Niveau : simple · Catégorie : Système
CPU, disque, processus, environnement.

3. Référence API

- exitOk(code number) → bool
- exitFailed(code number) → bool

4. Exemples d'utilisation

```
import "std/process"
use process
```

```
create result = exitOk(/* args */)
say result
```

```
create result = exitFailed(/* args */)
say result
```

5. Bonnes pratiques

- Préférez 'import "std/process"' en tête de fichier.
- Lancez 'afri-lang check' avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir /stdlib/ pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module process

```
function exitOk(code number) returns bool
end
```

```
function exitFailed(code number) returns bool
end
end
```

English

1. Overview

AFRILANG library 'std/process' (tier simple, category system). Exports 2 function(s): exitOk, exitFailed.

'import "std/process"' · 'use process'

```
- `exitOk(code number) → bool`
- `exitFailed(code number) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/process"
use process
```

Tier: simple · Category: System
CPU, disk, processes, environment.

3. API reference

- exitOk(code number) → bool
- exitFailed(code number) → bool

4. Usage examples

```
import "std/process"
use process
```

```
create result = exitOk(/* args */)
say result
```

```
create result = exitFailed(/* args */)
say result
```

5. Best practices

- Prefer `import "std/process"` at the top of your file.
- Run `afri-lang check` before shipping.
- Combine with related stdlib modules from the same category.
- See /stdlib/ for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module process

```
function exitOk(code number) returns bool
end
```

```
function exitFailed(code number) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/process"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/process/>

Source (extrait)

```
module process

function exitOk(code number) returns bool
end

function exitFailed(code number) returns bool
end
end
```